Assignment No	5
Title	Stacks :- 3B & 3C
Objective	Stack Application 1/2: Parenthesis Balancing, Postfix Evaluation
Roll No	MCA2649

Source Code:-

Stack Application 1: Parenthesis Balancing

```
//Name:- Satish Rathod
//Roll No:- 2649
//Module 3B:- Stacks
//Stack Application 1: Parenthesis Balancing


class ParBal
{
	public static void main(String[] args)
	{
		String expr = "((a+b)*(c+d))" ;

		if(isBalanced(expr))
		{
			System.out.println("the parenthesis are balanced in" + expr);
		}

		else
		{
			System.out.println("the parenthesis are not balanced" + expr);
```

```
        }
}//end

public static Boolean isBalanced(String ex)
{
        //Create an array with a tos to simulate a stack
        char[] stack = new char[ex.length()];
        int tos = -1;

        //scan the expression
        for(int i=0; i<ex.length(); i++)
        {
                char ch = ex.charAt(i);

                if(ch=='(')      //encounter an (push it on the stack
                {
                        tos++;
                        stack[tos]=ch;
                }
                else if(ch==')')  //encounter a ) pop the stack
                {
                        if(tos==-1) //no matching parenthesis
                        {
                                return false;
                        }
                        tos--;   //pop the stack
                }
        }// end of for i

        //If the stack is empty
```

```
            return tos==-1;

        }//end of  the isBalanced

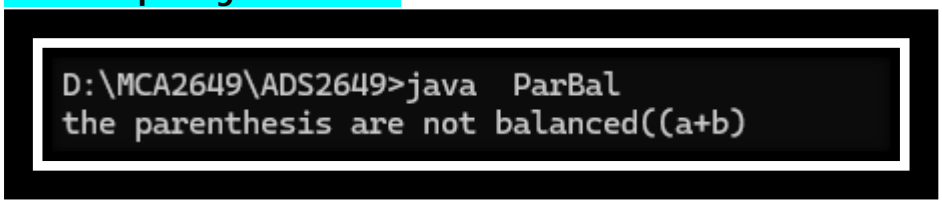

}//end
```

Output:-

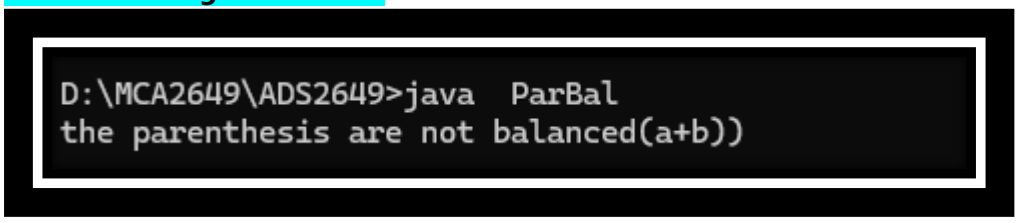
1. Balanced Parenthesis



2. Extra Opening Parenthesis



3. Extra Closing Parenthesis



Source Code:-

Stack Application of stack 2: Postfix Evaluation:-

```
//Name:- Satish Rathod

//Roll No:- 2649

//Module 3C:- Stacks

//Stack Application of stack 2: Postfix Evaluation
```

```
import java.util.*;

class PostEval
{
	public static void main(String[] args)
	{
		String expr = "23*5+62/-" ; //Test expression single digits
		int res = evaluatePostfix(expr);
		System.out.println("Result of evaluation of " + expr + " is: " + res);
	}//end of main

	public static int evaluatePostfix(String ex)
	{

		//Create a integer stack of TOS
		int[] stack = new int[ex.length()];
		int tos = -1;

		//Loop through each character of postfix expression
		for(int i = 0; i<ex.length(); i++)
		{
			char ch = ex.charAt(i);

			//If the character is a number push it on the stack
			if(Character.isDigit(ch))
			{
				tos++;
				stack[tos]= ch - '0'; // Convert char to int
			}
```

```
else if(ch=='+' || ch=='-' || ch=='*' || ch=='/')
{
        int op2 = stack[tos--];//Pop 2nd operand
        int op1 = stack[tos--];//Pop 1st operand

        int res = 0;
        switch(ch)
        {
                case '+' :
                                res = op1 + op2;
                                break;

                case '-' :
                                res = op1 - op2;
                                break;

                case '*' :
                                res = op1 * op2;
                                break;

                case '/' :
                                res = op1 / op2;
                                break;
        }//end of switch

        //push res back on the stack
        tos++;
        stack[tos] = res;
}//end of else
```

```
            }//end of for i
            //Final answer at the tos
            return stack[tos];

        }//end of evaluatePostfix

}//end of the class
```

Output:-



